# ParaDEVS Library

Build Report: what has been built so far, and how we know it works

ParaDEVS Library workspace, Binghamton

October 11, 2026 (covers work from October 9–11, 2026)

**How to read this report.** Page 3 is a one-page summary; if you read nothing else, read that. Every later section opens with a green "In short" box, so you can skim by reading only those boxes. Terms are defined in the glossary (Appendix B). The other documents are linked by name.

**ParaDEVS documents.** The entry point is **Start Here**, a reading guide that links every document.

- **Build Report** (this document): what has been built, how it was verified, what is next
- **How-To Guide**: setup, tutorials, API reference, troubleshooting
- **Architecture Guide**: how the library works and how to read results
- **RL Plan**: reinforcement-learning plan, analysis and results
- **Control Spec**: action selection: Para.decide, graph-mode planning, value and policy iteration
- **Hybrid Study**: which draws to enumerate or sample, and how cost grows with lookahead
- **Fast-Copy Proposal**: why the engine replays steps, where time goes, how to speed it up

## Contents

**1 At a glance** **3**

**2 Starting point** **3**

**3 What was built** **4**

3.1 How a model plugs in . . . . . 4

3.2 The parts, in plain terms . . . . . 5

3.3 The one idea that makes it general: branch at the draw . . . . . 5

3.4 Examples shipped with the library . . . . . 6

**4 How we know it works** **6**

4.1 Checks against independent references . . . . . 6

4.2 What merging saves . . . . . 7

**5 Problems found and fixed** **7**

**6 Documentation and working environment** **8**

**7 Reinforcement-learning track so far** **8**

7.1 The idea . . . . . 8

7.2 Setup (phase R0, done) . . . . . 8

7.3 First studies (phase R1, in progress) . . . . . 9

**8 Known limits of version 0** **9**

**9 What comes next** **10**

**10 Timeline** **11**

**A Reproducing every result** **11**

**B Glossary** **12**

**C Where things are** **13**

# 1 At a glance

| | |
|---|---|
| **What it is** | A Java library that runs an ordinary stochastic MS4 DEVS model as a *paratemporal* computation: each time the model makes a random choice, the engine follows the possible outcomes, weights them, and returns the exact (or sampled) distribution of what you asked it to observe. |
| **What a model must do** | One thing: get every random value from `Para.choose(label, law)` instead of a random number generator. Nothing else in the model changes, and it still runs in plain MS4. |
| **Does it work?** | **Yes. 77 of 77 checks pass** against three independent references: the paper's worked example (exact match), the Demo Deliverable finance run (to $10^{-12}$), and the original baseball engine (to $10^{-12}$). |
| **Speed-up from merging** | Baseball, 6 plate appearances: 24,450 leaves in 17.4 s without merging, 195 outcomes in 0.65 s with safe merging. Same answer. |
| **Documentation** | How-to guide with two tutorials (17 pp), architecture guide (13 pp), RL plan with results (9 pp), and this report. |
| **Working environment** | VS Code, preconfigured: one task builds and runs every test. |
| **RL track** | Setup done; Python can drive the Java engine. First studies: the paratemporal "exact" target roughly **halves the learning error per real episode** compared with standard n-step TD. |
| **Next** | Finish the Python prototypes (control, hybrid selection), then add RL features to the Java library and a Gymnasium wrapper for Python. |

**Decision needed**
Should the Java library get an *expectation-only* merge as an opt-in? It is much cheaper for RL targets, but it keeps only averages (not full distributions), which goes beyond the "safe merging only" rule you set. It is used only in the Python prototypes so far. Details in Section 9.

# 2 Starting point

**In short**
The old paratemporal engines worked, but only for one model shape, with the tree logic and domain code mixed together. The goal was a general library built on the theory in `Do s/documents/paratemporal`.

What existed on October 9:

- **Three copies of one hand-written engine**: Baseball v7, Finance v4-IH and the Demo Deliverable. Only the Demo Deliverable compiled and ran as-is; Finance v4-IH has an empty `CallGenerator.java`.
- **One model shape only**: one generator feeding one state machine, one random choice per step, made at a known time in a known component.
- **Workarounds in the wiring**: probabilities packed into message values, state written directly

into models, domain code (shares, runs) inside the engine.

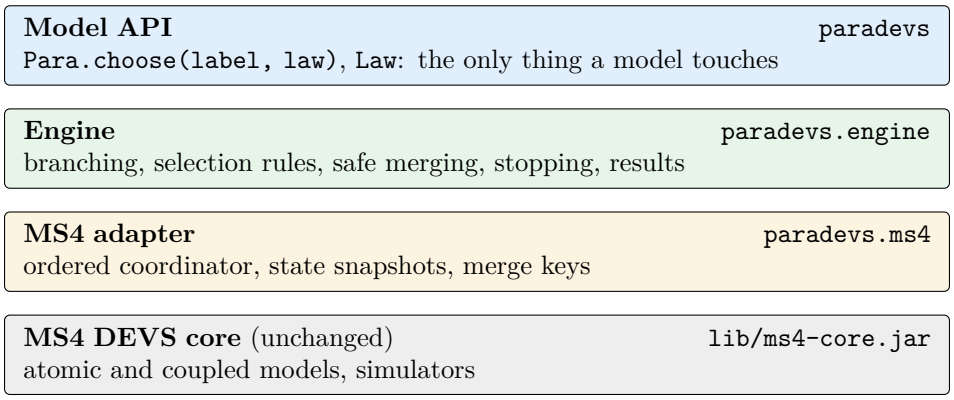
- **Step and draw were the same thing**: one tree level was one forced outcome, one full simulation run and one state save. A model with two random choices in one event could not be expressed.

Decisions you made (October 9), which the whole build follows:

1. Java, on top of the MS4 DEVS core.
2. The paper's conventions: models make draws inside their own transitions; the engine branches.
3. Safe merging only (the old IH "merge by shares" is not in the library).
4. Verification set: timed reward (paper), finance (Demo baseline), baseball (legacy engine).
5. Version 0 scope: fixed input trace, horizon counted in events, exhaustive / Monte Carlo / partial selection.
6. `Do s` is the theory authority; `Main_Sol_Base` is out of scope.

# 3 What was built

**In short**
Three small layers on top of the unchanged MS4 core: a one-call model API, the engine from the paper, and an adapter that drives MS4 models deterministically. About 1,200 lines of library code.




## 3.1 How a model plugs in

A model asks for random values like this. Under plain MS4 the call simply samples; under the engine it branches.

```
int phase = Para.choose("phase", Law.of(0, 0.25, 1, 0.75));   // 0 w.p. 0.25, 1 w.p.
    0.75
```

And an experiment runs it like this:

```
ParaResult<X> r = ParaEngine.run(
    ParaSpec.of(() -> new MyModel(), myTransducer)  // model + what to observe
```

```
        .trace(InputTrace.builder().at(0.0, "action", "a").build())
        .horizon(5)                                  // stop after 5 events
        .selection(SelectionRule.exhaustive())       // or monteCarlo(N), ...
        .merge(ParaSpec.Merge.SAFE));
r.distribution();  r.expectation(f);  r.residual();  r.stats();
```

### 3.2 The parts, in plain terms

| Part | What it does |
|---|---|
| `Para`, `Law` | The model's only contact with the library. `Law` is a finite list of values with probabilities; it rejects laws whose mass is not 1 (with `Law.asGiven` for slightly imperfect source data). |
| `ParaEngine` | The algorithm from `main_paratemporal.tex` (main / $\Gamma_{PT}$ / Advance / doExpansion / RunLocal), one global clock, stopping by horizon, acceptance or passivity. |
| `SelectionRule` | Which outcomes to follow at each draw: all (exhaustive), a sample (Monte Carlo), only likely ones (partial, with the leftover mass reported as the *residual*), or a mix chosen by draw label (hybrid). |
| `ParaSpec.Merge.SAFE` | Joins two branches only when *everything* about their future is identical: full model state, clock, inputs still to come and the observation so far. This cannot change any answer. |
| `ParaResult` | The distribution, expectations, marginals, total mass, residual and run statistics. |
| `ModelNode` | Drives MS4 atomic models through MS4's own simulator, and coupled models through a coordinator that visits children in a fixed order (MS4's own does not; see Section 5). |
| `Snapshots`, `StateKeys` | Copy a model's full state (Java serialization) and build its merge key (every field, by reflection, with an override hook). |

### 3.3 The one idea that makes it general: branch at the draw

The engine branches at each *draw*, not at each *event*. When a model calls `Para.choose` and the value is not yet known, the engine stops that event, creates one branch per outcome, and later replays the event from the saved state with the chosen value supplied. An event with two draws therefore becomes two levels of branching, which the old engines could not express. The architecture guide, Section "Step versus draw", explains this in detail.

### 3.4 Examples shipped with the library

| Example | What it shows |
|---|---|
| Tutorial: service line | The smallest complete model; answer checkable by hand (P(3 served) = 0.36). |
| Timed reward | The paper's worked example, with a known exact answer. |
| Finance | The Demo trader/portfolio rewritten as a plain DEVS coupled model, no engine-side domain code. |
| Baseball | The legacy batting model, same treatment. |
| Sandbox random walk | Template for new experiments (`src/sandbox`). |
| MS4 Me import demo | An unmodified MS4 Me-generated model (JobProcessorExample) running under the engine with two marked edits. |

## 4 How we know it works

**In short**
Every number the old engines and the paper produce is reproduced, exactly or to $10^{-12}$. All 77 checks pass in one command. Safe merging gives the same answers far faster.

### 4.1 Checks against independent references

| Test | Reference | Result | Checks |
|---|---|---|---|
| Tutorial | Hand calculation | P(3 served) = 0.36 exactly; Monte Carlo within 4 standard errors | 10 |
| Timed reward | `pt_reward_enumeration.py` from the paper | Exact law (6352, 10948, 9793, 12907)/40000; 64 leaves and 63 choice nodes as in the paper. Merging, sampling and partial runs all consistent | 24 |
| Finance | Demo Deliverable run of Oct. 9 | Share probabilities to $10^{-12}$; expected net $-38.9056847633$ vs $-38.9056847626$ | 28 |
| Baseball | Original BaseballDEVS_v7 engine, fresh run | All 39 (5 PA) and 55 (6 PA) states to $10^{-12}$; E[runs after 5 PA] = 0.315529858502410, identical | 15 |
| **Total (last run October 11, 2026)** | | | **77 / 77** |

Two more runs are part of the test script but have no numeric baseline: the sandbox random walk (binomial answer checked) and a plain MS4 run showing that models still work without the engine.

**Note**
The old IH numbers (merged by shares) are recovered as *marginals* of the exact distribution. That is why the library can drop IH merging without losing any result the old engine reported.

### 4.2 What merging saves

| Case | No merging | Safe merging (same answer) |
|---|---|---|
| Timed reward | 64 leaves | 8 outcomes |
| Finance, depth 6 | 729 leaves, 0.69 s | 288 outcomes, 0.35 s |
| Baseball, 5 PA | 4,890 leaves, 4.3 s | 145 outcomes, 0.41 s |
| Baseball, 6 PA | 24,450 leaves, 17.4 s | 195 outcomes, 0.65 s (legacy engine: about 17 s) |

## 5 Problems found and fixed

**In short**
Ten real problems came up. Two were genuine design hazards (nondeterministic MS4 ordering, and merging Monte Carlo samples). Both are now prevented in code and covered by tests.

| Problem | Effect | Fix |
|---|---|---|
| MS4's coupled coordinator visits children in hash order | A replayed event could make its draws in a different order, silently mixing up branches | Library ships its own coordinator with a fixed visiting order |
| Monte Carlo samples + safe merging | All samples start identical, so they merged into one path: P(+6) came out 0 instead of about 0.016 | Engine now refuses this combination with a clear message; test added |
| Baseball source data sums to 1.0004 (Machado's row) | Strict law check rejected the legacy data | `Law.asGiven(..., 10^-3)` for imperfect source data |
| Legacy baseball kept zero-probability states | State counts differed from the old engine | Compare positive-probability states only (the paper drops zero-weight branches) |
| Test helper compared `Long` with `Integer` | False failures | Compare numerically |
| Build script ignored compile errors | A broken build could look like a pass | Script now stops on any `javac` error |
| `javac` argument files split paths with spaces | Build failed in `OneDrive/Documents` | Quote each path |
| JPype returns map entries already unpacked | Python bridge script crashed | Use the unpacked pair directly |
| Long RL runs killed when memory was low | Two experiments lost partway | Experiments now save after each block and resume; run in the foreground |
| Demo notebook's RL reward | Reward is 1 exactly when the agent buys, whatever the price; the learned table carries no price information | Not fixed yet: reward must be redesigned before the finance RL problem (T5) |

# 6 Documentation and working environment

**In short**
Four PDFs, all LaTeX. VS Code is set up so that building, testing and recompiling documents are one-click tasks.

| Document | Read it when you want to... |
|---|---|
| **How-To Guide** (17 pp) | use the library: setup, a tutorial building a model in plain Java, a tutorial importing an MS4 Me model, API reference, troubleshooting. |
| **Architecture Guide** (13 pp) | understand how it is built, how results should be read, and what the theory guarantees. |
| **RL Plan** (9 pp) | see the reinforcement-learning plan, the merging-vs-variance analysis and the first results. |
| This report | see what exists, whether it works, and what comes next. |

**VS Code** (open the `ParaDEVS_Library` folder): Terminal → Run Task offers "build + run all tests", "run one test", the MS4 Me import demo, the RL tests and studies, and a compile task for each PDF. Run and Debug has entries for the sandbox, the tutorial and each verification test. Python uses the project environment `rl/.venv` automatically.

# 7 Reinforcement-learning track so far

**In short**
Python can now drive the Java engine. In the first studies, a paratemporal target (exact expansion with merging) learns about twice as accurately from the same real experience. Merging with a sample budget is exactly as good as independent sampling, no better and no worse.

## 7.1 The idea

In n-step learning, each update uses a guess of "what happens over the next $n$ steps". Standard n-step TD uses the one path the agent happened to take. With a simulator, the paratemporal engine can instead expand *all* paths from the current state (merging identical ones, which keeps it cheap) or sample several. The question is when this pays.

## 7.2 Setup (phase R0, done)

- Python environment `rl/.venv` with pinned Gymnasium, Stable-Baselines3, JPype, PyTorch and plotting.
- **Python ↔ Java works**: through JPype, Python ran the tutorial model in the Java engine and got the exact answer (0.36 / 0.64).
- The Demo's Q-learning notebook runs headless and reproduces the Demo Q-table within about 1%.

### 7.3 First studies (phase R1, in progress)

Two standard problems, four ways to form the n-step target: **A** standard n-step TD (one real path), **B** exact (exhaustive + merging), **C** sampled with merging and a budget of 4, **D** 4 independent rollouts.

| **Best error reached (lower is better)** | **T1: 19-state random walk** | **T2: FrozenLake 8×8** |
|---|---|---|
| A: standard n-step TD | 0.263 (work 99) | 0.161 (work 42) |
| B: exact, with merging | **0.139** (work 6,693) | **0.094** (work 21,502) |
| C: sampled + merging | 0.217 (work 3,268) | 0.124 (work 1,358) |
| D: independent rollouts | 0.218 (work 3,231) | 0.123 (work 1,227) |

Error = RMS error of the value estimate after the same number of real episodes. Work = simulated transitions per episode.

What this means:

1. **Same real experience, half the error.** B beats standard TD by about 2× on both problems.
2. **Merging a sampled tree ties with plain sampling** (C ≈ D everywhere), as the separate variance experiment predicted: merging saves work only if the freed budget is reallocated.
3. **The price is compute.** B spends far more simulation per episode. It is clearly worth it on the random walk; on FrozenLake (up to 12 next states per state) it is roughly break-even at equal compute.
4. **Where this helps**: when real experience is expensive and a simulator exists, which is exactly the DEVS setting.

Full tables, plots and the variance analysis are in the **RL Plan**.

## 8 Known limits of version 0

**In short**
The library is correct for what it covers, but it covers a deliberately small scope.

- **Inputs are a fixed trace.** Reactive inputs (an agent choosing actions during the run) are needed for RL and are planned (`Para.decide`, phase R2).
- **Draws inside output functions** work mechanically, but the deferred-output variant from the paper and its guarantees are not implemented.
- **Single-threaded.** Unmerged branches are independent, so parallel expansion is a natural next step.
- **Cost of copying.** Each replay restores a full serialized model; fine at these sizes, the first thing to optimise later.
- **Finite laws only** for exhaustive selection; continuous laws can only be sampled (as the paper says).

- **Not yet tried inside the MS4 Me IDE**: using `Para.choose` from DNL code, and copying the library into an MS4 Me project, are untested.
- **Generality not yet shown by an outsider**: no model has yet been written from scratch by someone other than the library author, following the guide.

## 9 What comes next

**In short**
Finish the Python prototypes, then bring the useful pieces into the Java library and wrap it for Python.

| **Phase** | Goal | Contents |
|---|---|---|
| **R1** | Finish Python prototypes | FrozenLake control (choosing actions, not only predicting values); hybrid selection (enumerate the slip, sample the policy); how fast exact expansion's cost grows with $n$ |
| **R2** | Java library features | $k$-sample Monte Carlo rule, per-stage budget, leaf-value hook, `Para.decide` for actions chosen during the run; expectation-only merge if approved |
| **R3** | Python wrapper | A Gymnasium environment (`ParaEnv`) over the Java engine; Java results must match the Python prototypes |
| **R4** | Baselines | Compare against tabular methods and Stable-Baselines3 (PPO, DQN, QR-DQN) |
| **R5** | Architecture | Read the DEVS message-broker paper (Dominici et al., SIMULTECH 2026) and decide between an in-process bridge, a service, or a broker |

**Decision needed**
**Expectation-only merge.** It joins branches that agree on everything that matters for future *averages*, keeping only expected values. It is exact for averages and makes RL targets much cheaper, but it discards the full distribution and so goes beyond "safe merging only". Options: (a) add it as a clearly labelled opt-in, off by default; (b) keep it in Python only. Recommendation: (a).

Open questions for the advisor and team (unchanged from the project notes):

1. Is a branch step tied to DEVS simulation time (as in the paper and the library) or to decision epochs (as in the old code)?
2. Is reduced-key (IH-style) merging needed at all, given that safe merging plus marginals reproduces the old IH numbers?
3. Who are the target users: MS4 Me modellers (DNL/SES extension) or Java programmers (library API)?

## 10 Timeline

| | |
|---|---|
| **Oct. 9, 2026** | Found the working baseline (Demo Deliverable) and reran it; reran Baseball v7 as a second baseline. Read the theory. Scope decisions. Built library v0 and ported all three examples; all checks pass. Wrote the How-To and Architecture guides; set up VS Code; verified the MS4 Me import demo. |
| **Oct. 10, 2026** | RL plan and merge-vs-variance experiment. R0 (Python environment, JPype bridge, notebook port). R1 prediction studies on T1 and T2; RL plan rewritten in LaTeX with results. |
| **Oct. 11, 2026** | This build report and the reading guide; full test suite rerun (77/77). |

## A Reproducing every result

From the `ParaDEVS_Library` folder in Git Bash (or the matching VS Code task):

| Command | What it reproduces |
|---|---|
| `bash scripts/build_library.sh` | Builds everything and runs all verification tests (Section 4) |
| `bash scripts/build_library.sh baseball` | One test: `tutorial`, `sandbox`, `reward`, `finance`, `baseball` or `plain` |
| `bash scripts/run_ms4me_import_demo.sh` | The MS4 Me import demo (needs MS4 Me installed) |
| `cd rl; .venv/Scripts/python.exe tests/test_targets.py` | Python estimator tests |
| `cd rl; .venv/Scripts/python.exe experiments/t1_nstep_td.py` | T1 study (resumes if interrupted; T2 is `t2_frozenlake_prediction.py`) |
| `rl/.venv/Scripts/python.exe rl/experiments/merge_variance.py` | The merge-vs-variance experiment |

# B Glossary

| | |
|---|---|
| **DEVS** | Discrete Event System Specification: a standard way to describe models that change state at events. |
| **Atomic / coupled model** | A single DEVS component / a network of components wired together. |
| **MS4 core, MS4 Me** | The Java DEVS simulator we build on / the IDE that generates MS4 Java code from SES/PES/DNL files. |
| **Paratemporal** | Following every possible outcome of the random choices in a simulation (or a sample of them) instead of one random run. |
| **Draw** | One call to `Para.choose`: a single random value with a known finite law. |
| **Branch** | One partial history being followed, with its probability weight. |
| **Leaf / outcome** | A finished branch. With merging, several leaves can be joined into one outcome. |
| **Choice node** | A point where a branch split at a draw. |
| **Departure** | One replay of an event from a saved state (the unit of work in the run statistics). |
| **Selection rule** | How the engine picks which outcomes of a draw to follow: all, a sample, the likely ones, or a mix. |
| **Safe merge** | Joining branches whose entire future is identical. It never changes an answer. |
| **Expectation-only merge** | Joining branches that agree on what matters for future averages. Exact for averages only. |
| **Residual** | Probability mass the engine chose not to follow (partial selection), reported so that bounds stay honest. |
| **Horizon** | How many events to simulate before stopping. |
| **n-step target** | In RL, the estimate of return over the next $n$ steps used to update a value estimate. |
| **Work** | Number of transitions simulated or examined; our measure of compute. |
| **JPype** | A Python package that runs Java in the same process; our Python $\leftrightarrow$ Java bridge. |

## C Where things are

| Path (in ParaDEVS_Library) | Contents |
|---|---|
| src/main/java/paradevs/ | The library (API, engine/, ms4/) |
| src/examples/java/examples/ | Tutorial, timed reward, finance, baseball, with their tests |
| src/sandbox/java/sandbox/ | Place for new experiments |
| src/ms4me_import_demo/ | MS4 Me-generated model under the engine |
| scripts/ | Build, test and run scripts |
| docs/ | All PDFs; start at docs/ParaDEVS_Start_Here.pdf |
| rl/ | Python: prototypes, experiments, tests, baselines, environment |
| runs/ | Saved baseline runs of the old engines |
| notes/ | Working notes from the first days (Markdown, superseded by the PDFs) |